

Array

Array is a linear collection of similar elements.
Array is also known as subscript variable.

- Array ^{data dalaat} Chahiye Jo same data-type ho or same nature ke ho.
Write a program to calculate average of 100 numbers.
→ we can perform the above task by our regular method but it would be difficult to handle so many variables in the program and the program would become very lengthy.

So for these type of problem the concept of array is used.

Declaration of 1-D array

data type array name [size]; ^{square} bracket

array name — denotes the name of the array and it can be any valid C identifier

data type — is the data type of the element which will store in array

[size] — The size of the array specifies the number of elements that can be stored in the array.

Ex - `int age[95];`

Here age is an array of type int, which can store 100 elements of type int.

index ←

0 1 2 3 4 5 6 7 8 9

$a[3] = 25;$

[] $\rightarrow$ subscript operator

Array declaration rules

↑ can't be empty

Define total number of variables in array

it is not an index

0	1	2	3	4

whatever is the size of an array
it always consumes memory in a
sequential order & continuous memory
location.

local array (array declared under an function) * when not initialized contains garbage values

4. You can initialize array during declaration

`int a[5] = {10, 50, 30, 70, 20};`

0	1	2	3	4
10	50	30	70	20

5. You cannot initialize an array during declaration more than its size

`int a[5] = {10, 50, 30, 70, 20, 80, 40};`

Error

6. You can initialize an array during declaration with lesser values than the size of an array

`int a[5] = {10, 50};`

0	1	2	3	4
10	50	0	0	0

*

Remaining variables in array will contain 0 not a garbage value.

7. During declaration you can leave `[]` empty only when you initialize array at the same time.

`int a[] = {10, 50, 30, 80, 20};`

2-D Array →

Array of Array

Two dimensional array is also known as matrix.

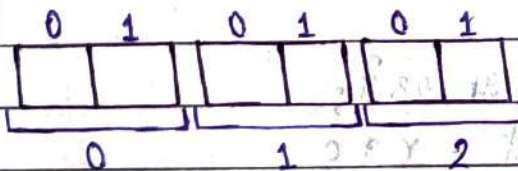
data type array name [r][c];

This is a 2D array where r depicts number of rows in matrix and c depicts number of columns in the matrix.

Declaration of a 2-Dimensional Array

int arr[3][2];

arr[r][c];



arr[0][0]	arr[0][1]
arr[1][0]	arr[1][1]
arr[2][0]	arr[2][1]

while initializing a 2D array it is necessary to mention the second (column) dimension, whereas the first dimension (row) is optional.

Correct ÷

int arr[2][3];

int arr[][3];

Incorrect :

`int arr [2][7];` `int arr [7][7];`

Initialisation of a 2-Dimensional Array

`int arr [4][2] = { {1234, 56}, {1256, 43}, {1434, 32}, {1312, 96} };`

$\begin{matrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ \hline 0 & 1 & 2 & 3 \end{matrix}$

`int arr [4][2] = { 1234, 56, 1256, 43, 1434, 32, 1312, 96 };`

$\begin{matrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ \hline 0 & 1 & 2 & 3 \end{matrix}$

Compiler automatically assign the value in particular row and column in sequential order

`int arr [7][3] = { 12, 34, 56, 78, 91, 23 };`

Total no. of element = $r * c$

$* 3 = 2$

`int arr [7][3] = { 12, 34, 56, 78, 91, 23 };`

$\begin{matrix} 0 & 1 & 2 & 0 & 1 & 2 \\ \hline 0 & 1 \end{matrix}$

	0	1	2
0	12	34	56
1	78	91	23

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int a[2][3] = { 5, 8, 9, 10, 12, 11 };
```

```
    int i, j;
```

```
    for (i = 0; i < 2; i++)
```

```
    {
```

```
        for (j = 0; j < 3; j++)
```

```
        {
```

```
            printf("%d", a[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
}
```

Output :-

5 8 9

10 12 11

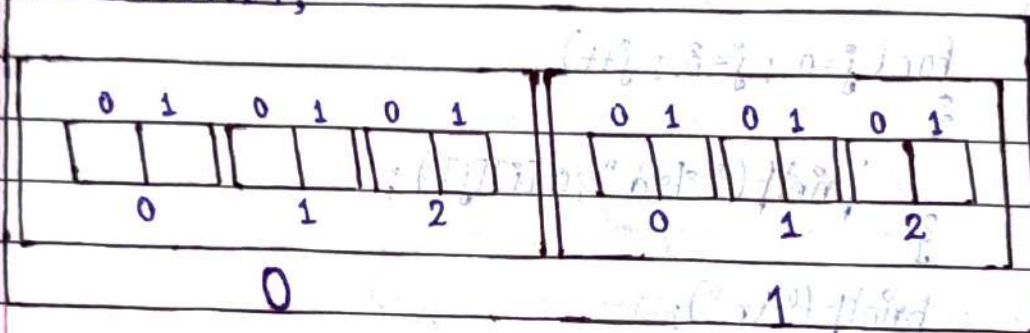
3-D Array

Array of 2-D Array

Array of Array of Arrays.

data type array name [size][size][size];

int [2][3][2];



0 1 2

0 1 2

0 1 2		0 1 2	
0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0

Correct :-

a[2][3][2] ✓

Initialisation of a 3-Dimensional Array

int a[2][3][2] = { { 4, 5, 6, 7, 8, 9 }, { 10, 11, 12, 13, 14, 15 } }

int a[2][3][2] = { 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 }

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[2][3][2] = { 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 }
```

```
    int i, j, k;
```

```
    for(i=0; i<2; i++)
```

```
    {
```

```
        for(j=0; j<3; j++)
```

```
        {
```

```
            for(k=0; k<2; k++)
```

```
            {
```

```
                printf("dod " a[i][j][k]);
```

```
            }
```

```
        printf("\n");
```